

NoraApp

AI-Powered Focus & Productivity Platform

Comprehensive Project Report

Architecture | Features | AI System | Backend | UI/UX Design

Generated: September 11, 2026

Version 1.0

Table of Contents

1. Executive Summary
2. Project Overview
3. System Architecture
4. Tech Stack
5. Free Infrastructure Stack (\$0/month)
6. Adaptive Persona System (4 Age Groups)
7. Feature Modules (15 Screens)
8. AI System & Digital Assistant
9. Backend API
10. Services Layer (Flutter)
11. UI/UX Design System
12. Data Models
13. Project File Structure
14. How It Works End-to-End
15. Business Model
16. Future Roadmap

1. Executive Summary

NoraApp is an AI-powered focus and productivity mobile application designed to combat doomscrolling and social media addiction. It converts idle time into micro-learning and productive habits through an intelligent adaptive system.

The app features a unique adaptive persona system with four age groups (Baby 1-6, Kid 6-12, Teen 12-18, Adult 18+), each receiving a personalized AI personality, UI theme, content strategy, and interaction style. Nora uses a Unified LLM Provider that auto-falls back across Groq API (fast, free), Ollama local, and Colab GPU (heavy models) for privacy-first AI inference. The backend runs on PocketBase (Oracle Cloud free tier) and Firebase Crashlytics for crash reporting -- all at zero cost.

The architecture is a Flutter frontend communicating with a Python FastAPI backend. The backend integrates with a unified LLM provider (Groq + Ollama + Colab), SQLAlchemy for database persistence, and JWT for authentication. The frontend uses Provider for state management and platform channels for Android-native device integration.

Key Metrics

Screens: 15 feature modules

Services: 6 Flutter services + 5 backend services

AI Providers: Groq API (fast) + Ollama (local) + Colab GPU (heavy)

Backend Endpoints: Auth, Chat, Command, App Classification, Health, AI Chat

Target Platforms: Android (primary), iOS (planned)

Monthly Cost: \$0 -- no credit cards, no surprise bills

Design Philosophy: Dark mode, minimalist, neptun green & purple neon accents

2. Project Overview

Mission

Combat doomscrolling and social media addiction by converting idle time into micro-learning and productive habits. Nora is not just a timer or a blocker -- it is an intelligent companion that understands the user's age, habits, and goals, then adapts its behavior accordingly.

Target Audience

- ADHD users who need structured focus support
- Students preparing for exams and managing study time
- Freelancers and remote workers seeking productivity routines
- Corporate employees managing digital wellness
- Parents wanting age-appropriate screen time management for children

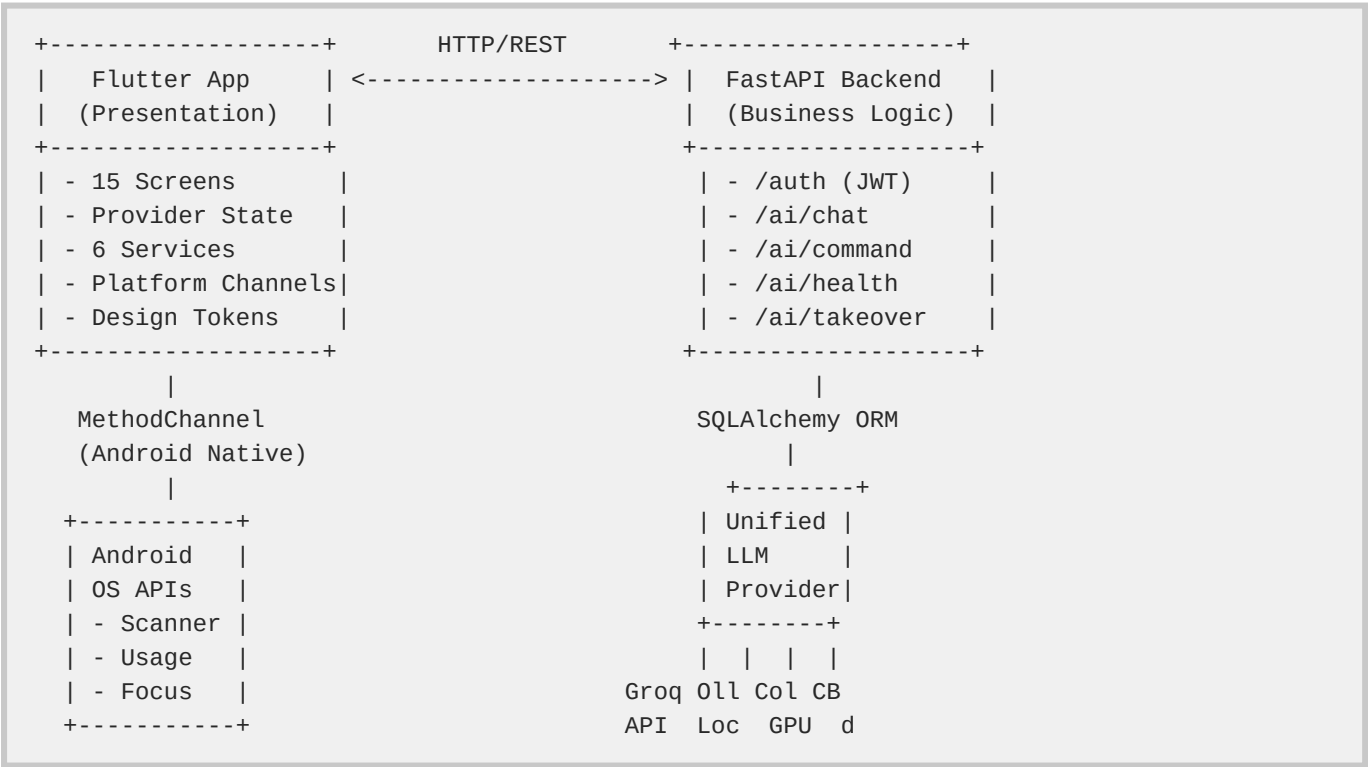
Core Value Proposition

Unlike generic screen time trackers, Nora provides: (1) Age-adaptive AI personalities that change language, complexity, and tone based on the user's age group; (2) Real device control via Android accessibility services for actual app blocking; (3) Privacy-first AI running locally via Ollama with no cloud dependency; (4) A complete productivity loop: plan -> focus -> reflect -> review.

3. System Architecture

NoraApp follows a clean three-tier architecture: Presentation (Flutter/Dart), Business Logic (Services + Provider), and Data (FastAPI + SQLAlchemy + Unified LLM Provider). The frontend communicates with the backend via REST API over HTTP.

Architecture Diagram (Text)



Communication Flow

1. User interacts with a Flutter screen (e.g., sends a chat message). 2. The screen calls a Flutter service (e.g., LlmService). 3. The service sends an HTTP request to the FastAPI backend. 4. The backend processes the request, optionally querying Ollama for LLM inference. 5. The response flows back through the same chain. 6. For device operations (app scanning, blocking), Flutter uses MethodChannel to call Android native code directly.

4. Tech Stack

Frontend (Flutter)

Framework: Flutter 3.x (Dart)
State Management: Provider (ChangeNotifier pattern)
HTTP Client: package:http (for REST API calls)
Platform Integration: MethodChannel (Android native bridge)
SVG Rendering: flutter_svg
Local Storage: SharedPreferences + SQLite
Animations: Lottie + Custom AnimationController
Typography: SF Pro Display + Inter (custom fonts)
Design System: Custom DesignTokens + PersonaTheme

Backend (Python)

Framework: FastAPI 0.104.1
ASGI Server: Uvicorn 0.24.0
ORM: SQLAlchemy 2.0.23
Database: SQLite (dev) / MariaDB / PocketBase (prod)
Authentication: JWT (python-jose) + bcrypt password hashing
AI/LLM: Unified LLM Provider: Groq API + Ollama + Colab GPU
Validation: Pydantic 2.5.2
HTTP Client: httpx 0.25.2 (for Groq + Ollama communication)
Crash Reporting: Firebase Crashlytics (unlimited, \$0)

Android Native

Language: Kotlin (via Flutter MethodChannel)
App Scanning: PackageManager API
Usage Tracking: UsageStatsManager API
App Blocking: AccessibilityService + DevicePolicyManager
Permissions: Usage Access, Accessibility, Query All Packages

5. Free Infrastructure Stack (\$0/month)

NoraApp runs on a complete zero-cost infrastructure stack. No credit cards are required for any component. There are no surprise bills, no vendor lock-in, and no risk of unexpected charges. All services use free tiers that are permanently free, not trial periods.

LLM Inference Providers

Provider	Type	Free Limit	CC?	Role
Groq API	Fast LLM inference	30 RPM, 14400 RPD	No	Primary (fastest)
Ollama Local	Self-hosted LLM	Unlimited	No	Fallback 1
Colab GPU	Heavy models via T4	~12h/day	No	Fallback 2 (GPU)

Infrastructure Services

Service	Provider	Free Limit	CC?	Purpose
PocketBase	Oracle Cloud	4 OCPUs, 24GB RAM	No	Backend/DB
Crashlytics	Firebase	Unlimited events	No	Crash reporting
MariaDB	Self-hosted	Unlimited	No	Primary DB

Unified LLM Provider

The UnifiedLLMProvider class in `backend/ai/llm_provider.py` automatically selects the best available LLM provider with automatic fallback:

- Priority 1: Groq API -- fastest response times (30 requests/minute free)
- Priority 2: Ollama Local -- runs on localhost:11434 (unlimited)
- Priority 3: Colab GPU -- connects via ngrok tunnel to T4 GPU (15GB VRAM)

Set `LLM_PROVIDER=auto` in `.env` to enable automatic fallback. The provider tries Groq first, falls back to Ollama local, then to Colab GPU. All existing code that uses `from ai.ollama_client import ollama` continues to work unchanged.

Total Monthly Cost

Groq API	\$0	(free tier, 30 RPM)
Colab GPU (T4)	\$0	(free when running)
Ollama Local	\$0	(runs on your machine)
PocketBase (Oracle)	\$0	(4 0CPUs, 24GB RAM forever)
Firebase Crashlytics	\$0	(unlimited events)

TOTAL	\$0/month	

6. Adaptive Persona System

Nora's defining feature is its adaptive persona system. Based on the user's age group, the entire app experience changes: AI personality, language complexity, UI colors, mascot, content strategy, and even the max number of daily tasks. The system is defined in the AgeGroup enum and exposed via PersonaTheme.

Group	Personality	Language	Topics	Tasks	UI Style
Baby (1-6)	Warm, gentle	Simple words, emojis	Colors, shapes, numbers	1	Bright pastels
Kid (6-12)	Fun, encouraging	Enthusiastic, simple	Homework, trivia, stories	2	Vibrant, gamified
Teen (12-18)	Mentor-like, real	Modern, no lectures	Study, mental health	3	Sleek dark theme
Adult (18+)	Professional	Concise, frameworks	Deep work, GTD, habits	3	Dark minimalist

Persona Configuration per Age Group

- Baby: mascotName='Nora', mascotEmoji='star', primaryColor=neptun green, maxDailyTasks=1
- Kid: mascotName='Nora', mascotEmoji='rocket', primaryColor=blue, maxDailyTasks=2
- Teen: mascotName='Nora', mascotEmoji='bolt', primaryColor=purple, maxDailyTasks=3
- Adult: mascotName='Nora', mascotEmoji='diamond', primaryColor=cyan, maxDailyTasks=3

Each persona has a unique system prompt that controls the LLM's personality, vocabulary, response length, and topic focus. The prompt changes dynamically based on the age group, ensuring age-appropriate interactions at all times.

7. Feature Modules (15 Screens)

Splash

Description: App loading screen with brand animation

File: splash_screen.dart

Welcome

Description: First-time user greeting and onboarding start

File: welcome_screen.dart

Onboarding

Description: Multi-step persona selection, permissions setup

File: onboarding_screen.dart

Auth

Description: Login/Register with JWT, age group selection

File: auth_screen.dart

Home

Description: Age-adaptive dashboard: header, insights, stats, actions, motivation

File: home_screen.dart

Chat

Description: AI chat with Nora using Ollama LLM, age-group personalities

File: chat_screen.dart

Assistant

Description: Digital assistant with device control (scan, block, track, focus)

File: assistant_screen.dart

Scan

Description: AI-powered app scanning with classification and blocking recommendations

File: app_scan_screen.dart

Timer

Description: Pomodoro-style focus timer with blocking integration

File: timer_screen.dart

Plan

Description: Daily planning: morning tasks, active day tracking, evening reflection

File: plan_screen.dart

Weekly Review

Description: Comprehensive weekly reflection: goals, mood, AI insights, visualization

File: weekly_review_screen.dart

Breathing

Description: Guided breathing exercises for stress relief and focus

File: breathing_screen.dart

Stats

Description: Usage statistics, streaks, achievements, analytics

File: stats_screen.dart

Profile

Description: User profile, settings, persona info, streaks

File: profile_screen.dart

Feed

Description: AI-curated learning content feed

File: feed_screen.dart

Feature: Home Screen

The Home screen is the main dashboard, composed of five widgets: HomeHeader (greeting, streak, persona mascot), HomeInsight (AI-generated smart summary of today's usage), HomeStats (focus minutes, tasks completed, streak), HomeActions (quick action buttons to start focus, scan apps, chat), and HomeMotivation (age-appropriate motivational quote).

Feature: Chat Screen

Full chat interface for talking to Nora AI. Uses LlmService to send messages to the backend, which forwards to Ollama. Maintains conversation history (last 10 messages for context). Shows backend connection status (Online/Offline). Persona-adaptive: different mascot emoji, greeting, and personality per age group.

Feature: Digital Assistant

A more advanced chat interface with real device control capabilities. The AI can execute JSON actions: scan_apps, block_apps, unblock_apps, start_focus, show_usage, show_recommendations. Includes quick action chips for common operations. Builds device context (blocked apps, usage data, scan results) and sends it with each command for context-aware AI responses.

Feature: App Scanner

Tabbed interface (All Apps / AI Recommend / Blocked) for managing device applications. Scans all installed apps via Android MethodChannel, merges usage data, sends to backend AI classifier for categorization, then displays results with real app icons loaded in parallel batches. Users can toggle block status per-app or apply all AI recommendations.

Feature: Daily Plan

Four-state adaptive flow: (1) Morning Planning -- user adds 1-3 tasks with priorities (Must do / Should do / Nice to do); (2) Active Day -- progress ring, task completion tracking, points earned; (3) Evening Reflection -- day summary with stats, journaling prompt; (4) History View -- past plans with completion rates and reflection status.

Feature: Weekly Review

Comprehensive weekly reflection with animated entrance. Sections: WeeklySummaryCard (total focus, sessions, points, streak, days active), MoodSelector (emoji mood picker), GoalsSection (add/track weekly goals with progress bars), ReflectionsSection (age-adaptive prompts), AiInsightCard (generated insights), WeekVisualization (bar chart of daily focus minutes). All content adapts labels and complexity per age group.

8. AI System & Digital Assistant

Nora's AI system is the core intelligence layer. It operates in two modes: Chat Mode (conversational AI for Q&A and guidance) and Command Mode (digital assistant with device control capabilities). Both modes use age-group-specific system prompts.

Chat Mode (`LlmService.sendMessage`)

Sends user messages to `/ai/chat` endpoint. Backend constructs a system prompt based on age group (baby/kid/teen/adult), appends conversation history, and forwards to Ollama. Returns the AI response text. Used in ChatScreen.

Command Mode (`LlmService.sendCommand`)

Sends commands to `/ai/command` endpoint with device context (scan results, blocked apps, usage data). Backend uses assistant-specific prompts that include device control instructions. The AI can return JSON actions that are parsed and executed by the frontend.

AI Actions (JSON Protocol)

- scan_apps:** Triggers app scanning via MethodChannel
- block_apps:** Blocks specified packages (requires confirmation)
- unblock_apps:** Unblocks specified packages
- start_focus:** Starts a focus session with specified duration (minutes)
- show_usage:** Displays today's usage statistics
- show_recommendations:** Shows AI blocking recommendations
- analyze_usage:** Analyzes usage patterns for insights

Unified LLM Provider Integration

The backend communicates with multiple LLM providers via the `UnifiedLLMProvider` class. It automatically selects the best available provider: Groq API (fastest, free tier), Ollama local (localhost:11434), or Colab GPU (via ngrok tunnel). The system sends chat completion requests with age-group system prompts and user messages. The LLM processes the request and returns a response. All providers are free with no credit card required.

App Classification AI

When apps are scanned, the backend uses AI to classify them into categories (social_media, entertainment, games, productivity, messaging, education, finance, health, etc.). It also generates blocking recommendations based on the user's age group. For example, a Teen user might get social media apps recommended for blocking

during study time.

9. Backend API (FastAPI)

The backend is a Python FastAPI application with SQLAlchemy ORM, JWT authentication, and Ollama LLM integration. It runs on Uvicorn and supports SQLite (development) or PostgreSQL (production).

API Endpoints

POST /auth/register: Register new user (email, password, age_group)

POST /auth/login: Login with email/password, returns JWT token

GET /auth/me: Get current user profile (requires JWT)

POST /ai/chat: Send chat message to Nora AI (message, age_group, history)

POST /ai/command: Send command to digital assistant (command, age_group, context)

GET /ai/health: Check if Ollama backend is running

POST /ai/classify: Classify apps by category and generate blocking recommendations

Database Schema

The backend uses SQLAlchemy ORM with the following core models: User (id, email, hashed_password, age_group, persona_name, streak_days, total_points, created_at), TimerSession (id, user_id, duration_minutes, completed, blocked_apps_json, created_at), ContentItem (id, title, body, category, difficulty, age_group, created_at), Achievement (id, user_id, type, title, description, points, unlocked_at).

Authentication Flow

1. User sends POST /auth/register with email, password, age_group. 2. Backend hashes password with bcrypt, stores user in database. 3. User sends POST /auth/login with credentials. 4. Backend verifies password, generates JWT token (python-jose) with user ID and expiry. 5. Flutter stores token in SharedPreferences. 6. All subsequent API calls include Authorization: Bearer <token> header.

10. Services Layer (Flutter)

LlmService

Connects to backend /ai/chat and /ai/command endpoints. Sends messages, receives responses, checks Ollama availability. Handles both chat and command modes.

AppScannerService

Android MethodChannel bridge for scanning all installed apps, getting app details, managing blocked apps list, and retrieving app icons as base64.

FocusProtectionService

Android MethodChannel bridge for focus mode protection. Manages authorization, blocking enable/disable, app selection, and status monitoring.

UsageTrackerService

Tracks screen time per app, provides daily/weekly usage summaries, categorizes usage by app type (social, entertainment, productivity).

ApiService

General HTTP API client for backend communication. Handles auth tokens, app classification requests, and other API operations.

ProactiveAssistService

Provides proactive AI-powered insights. Generates smart summaries of today's usage, suggests actions, and provides motivational content.

11. UI/UX Design System

Design Philosophy

NoraApp follows Apple Human Interface Guidelines with a dark-mode-first approach. The visual identity is built around neptun green (#00C896) and purple neon accents, creating a modern, calming, and focused aesthetic. Every element is designed to reduce visual noise and support concentration.

Design Tokens (design_tokens.dart)

Background: #0A0E1A (deep dark)
Surface: #141927 (card background)
Surface Raised: #1C2235 (elevated elements)
Accent Primary: #00C896 (neptun green)
Accent Secondary: #7B61FF (purple neon)
Accent Tertiary: #FF6B9D (pink accent)
Text Primary: #FFFFFF
Text Secondary: #B0B8C8
Text Muted: #6B7280
Success: #00C896
Warning: #FFB800
Danger: #FF4757
Font Family Primary: Inter
Font Family Display: SF Pro Display

Persona-Based Theming

PersonaTheme generates complete theme configurations per age group. Each persona defines primary/secondary/tertiary colors, mascot name and emoji, greeting messages, plan titles, reflection prompts, and max daily tasks. The theme is applied app-wide via Provider.

Shared Components (NoraComponents)

- NoraButton -- Gradient primary button with icon, loading state, expansion
- NoraCard -- Themed card with optional gradient, border, padding
- NoraMascot -- Animated mascot display with glow effect
- SectionHeader -- Section title with icon and label
- PlanPromptCard -- Onboarding plan prompt with mascot illustration

- PlanTaskCard -- Task card with checkbox, priority, completion animation
- PlanProgressRing -- Circular progress indicator for task completion

12. Data Models

User

id, email, name, ageGroup, personaName, streakDays, totalPoints, createdAt, lastActiveAt

TimerSession

id, userId, durationMinutes, completed, blockedAppsJson, createdAt

ContentItem

id, title, body, category, difficulty, ageGroup, createdAt

Achievement

id, userId, type, title, description, points, unlockedAt

AppInfo

packageName, appName, category, isSystemApp, isBlocked, aiRecommendedBlock, usageTodayMinutes

DailyPlan

date, tasks(List), allCompleted, pointsEarned, eveningReflected

WeeklyReview

weekStart, totalFocusMinutes, totalSessions, totalPointsEarned, streakDays, mood, goals, reflections, aiInsight

ChatMessage

role (user/assistant), content

LlmResponse

response, model, error

AssistantResponse

response, model, actions(List<AIAction>), error

AIAction

type, description, data(Map)

FocusProtectionStatus

supported, authorized, usageAccessGranted, accessibilityGranted, blockingEnabled, blockedApps, message

UsageStatsSummary

totalScreenTimeMinutes, socialMediaMinutes, entertainmentMinutes, productivityMinutes, appCount, topApps

13. Project File Structure

```

NoraApp/
  lib/
    main.dart                # App entry, routes, providers
    core/
      constants/design_tokens.dart  # UI design tokens
      enums/age_group.dart          # 4 age groups + persona mapping
      theme/
        app_theme.dart             # Light/Dark theme
        persona_theme.dart         # Persona-based themes
    models/
      models.dart                # User, TimerSession, ContentItem, Achievement
      app_info.dart              # AppInfo model
    providers/
      app_provider.dart           # Central state (persona, auth, timer, etc.)
    services/
      api_service.dart            # HTTP API client
      llm_service.dart            # LLM integration (chat + command)
      app_scanner_service.dart     # Android app scanning bridge
      focus_protection_service.dart # Focus mode protection bridge
      usage_tracker_service.dart   # Screen time tracking
      proactive_assist_service.dart # AI-powered insights
    features/
      home/screens/home_screen.dart
      chat/screens/chat_screen.dart
      chat/screens/assistant_screen.dart
      scan/screens/app_scan_screen.dart
      plan/screens/plan_screen.dart
      weekly_review/screens/weekly_review_screen.dart
      timer/, auth/, onboarding/, breathing/,
      stats/, profile/, feed/, splash/, welcome/
    widgets/
      nora_components.dart        # Shared UI components
  backend/
    main.py                      # FastAPI entry point
    .env                          # GROQ_API_KEY, LLM_PROVIDER
    ai/
      llm_provider.py             # Unified Groq/Ollama/Colab provider
      ollama_client.py            # Backward-compatible wrapper
      prompts.py                  # Age-group system prompts
    api/                          # Route handlers
    services/
      agent_capabilities.py        # AI agent capabilities
      app_classifier.py            # App classification logic
    models/                       # SQLAlchemy ORM models
    utils/                         # Utility functions
    requirements.txt              # Python dependencies
  free-stack/
    README.md                     # Full documentation
    QUICKSTART.md                 # 5-minute setup guide
    groq/                         # Standalone Groq scripts
    colab-gpu/                    # Colab notebook + client
    pocketbase/                   # Oracle Cloud setup
    crashlytics/                  # Firebase integration
    config/providers.json          # Provider registry

```

14. How It Works End-to-End

User Journey

- 1. Install & Open: User installs NoraApp, sees splash screen with brand animation.
- 2. Onboarding: Selects age group, grants permissions (Usage Access, Accessibility).
- 3. Registration: Creates account or logs in (JWT authentication).
- 4. Home Dashboard: Sees personalized dashboard with stats, quick actions, motivation.
- 5. App Scanning: Scans all installed apps, AI classifies and recommends blocks.
- 6. Focus Session: Starts timer, blocking apps are activated via accessibility service.
- 7. AI Chat: Talks to Nora for guidance, study tips, or just conversation.
- 8. Digital Assistant: Uses voice-like commands to scan, block, track, and focus.
- 9. Daily Planning: Sets morning tasks, tracks completion, reflects in evening.
- 10. Weekly Review: Reviews weekly progress, sets goals, gets AI insights.

Focus Session Flow

```
User taps 'Start Focus' -> Timer starts -> FocusProtectionService.enableBlocking()  
-> MethodChannel calls Android AccessibilityService  
-> Blocked apps trigger overlay when opened  
-> Timer completes -> Stats updated -> Points earned  
-> Streak checked -> Achievements potentially unlocked
```

AI Chat Flow

```
User types message -> LlmService.sendMessage()  
-> HTTP POST to backend /ai/chat  
-> Backend constructs prompt (age-group system + history)  
-> Forwards to Ollama LLM (local inference)  
-> Response returned to Flutter -> Displayed in chat UI
```

15. Business Model

B2C Freemium

- Free Tier: Basic timer, manual app blocking, limited AI chat (5 messages/day)
- Premium (\$4.99/mo): Unlimited AI chat, smart blocking, weekly reports, custom themes
- Family (\$9.99/mo): Up to 5 family members, parental controls, child personas

B2B (Companies/Universities)

- Enterprise Dashboard: Team productivity analytics, anonymized aggregate data
- Per-seat pricing: \$2.99/user/month for corporate wellness programs
- University licenses: Bulk pricing for student digital wellness initiatives

Revenue Projections

Year 1 target: 50K free users, 5K premium subscribers, 2 enterprise clients. Projected ARR: \$300K-\$500K. Break-even expected at 15K premium subscribers.

16. Future Roadmap

Phase 1 (Current): Core app with AI chat, app scanning, focus timer, daily planning

Phase 2 (Q4 2026): iOS support (Screen Time API), cloud sync, social features, achievement system

Phase 3 (Q1 2027): Wearable integration (Apple Watch, Wear OS), calendar sync

Phase 4 (Q2 2027): Enterprise dashboard, team analytics, API for third-party integrations

Phase 5 (H2 2027): Voice assistant mode, AR overlays, predictive focus scheduling

End of Report